

Implementazione Feature: Prenotazione Tavolo

Panoramica

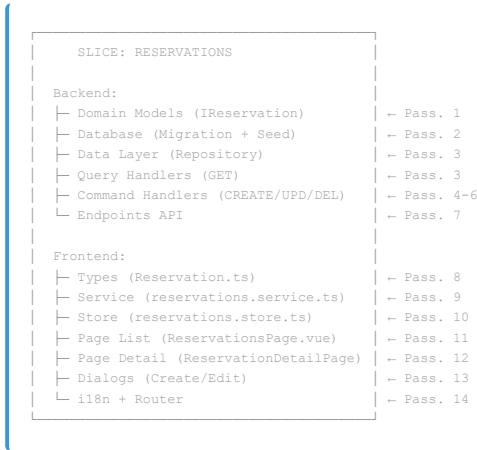
Questa documentazione guida l'implementazione **step-by-step** della feature "Prenotazione Tavolo" seguendo l'architettura **Slice Vertical** del progetto.

Cosa crea l'utente: - Una sezione di gestione prenotazioni tavoli - Una tabella con 2 campi principali: `utente` e `numero tavolo` - Gestione completa: lista, creazione, modifica, eliminazione - Integrazione backend + frontend

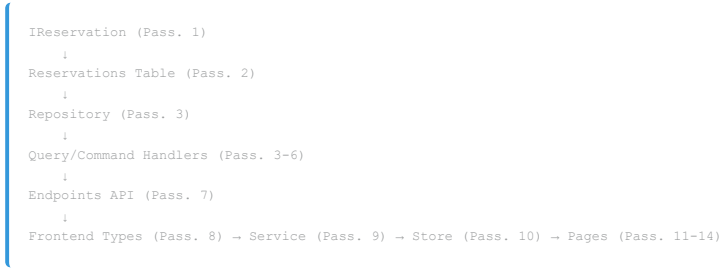
Numero di passaggi: 13 passaggi

Architettura Slice Vertical — Panoramica

La **Slice Vertical** isola per dominio, non per layer:



Flow di dipendenze:



BACKEND — 7 Passaggi

Passaggio 1: Aggiungere l'Interfaccia e il Domain Model

Cosa fare

Creare i file che definiscono il **modello di dominio** per una prenotazione:

1. File: `IReservation.cs` — Interfaccia del dominio
2. Proprietà: `Id`, `UserId`, `TableNumber`, `ReservationDate`, `Status`, `CreatedAt`, `UpdatedAt`
3. Nessuna logica, solo proprietà pubbliche
4. File: `ReservationRecord.cs` — Implementazione concreta

5. È un `record` (immutable) che implementa `IReservation`
6. File: `ReservationStatus.cs` — Enum per lo stato
7. Valori: `Pending`, `Confirmed`, `Cancelled`
8. File: `ReservationValidator.cs` — Validatore FluentValidation
9. Tavolo deve essere tra 1-20
10. Data non nel passato
11. UserId non vuoto

File coinvolti

```
Backend/Slices/Reservations/
├── Domain/
│   ├── IReservation.cs          ← NEW
│   ├── ReservationRecord.cs     ← NEW
│   └── ReservationStatus.cs     ← NEW (enum)
└── Validators/
    └── ReservationValidator.cs  ← NEW
```

Codice di Esempio

```
// 📄 Domain/IReservation.cs
public interface IReservation
{
    string Id { get; }
    string UserId { get; }
    int TableNumber { get; }
    DateTime ReservationDate { get; }
    ReservationStatus Status { get; }
    DateTime CreatedAt { get; }
    DateTime UpdatedAt { get; }
}

// 📄 Domain/ReservationStatus.cs
public enum ReservationStatus
{
    Pending = 0,
    Confirmed = 1,
    Cancelled = 2
}

// 📄 Domain/ReservationRecord.cs
public record ReservationRecord(
    string Id,
    string UserId,
    int TableNumber,
    DateTime ReservationDate,
    ReservationStatus Status,
    DateTime CreatedAt,
    DateTime UpdatedAt
) : IReservation;

// 📄 Validators/ReservationValidator.cs
using FluentValidation;

public class ReservationValidator : AbstractValidator<IReservation>
{
    public ReservationValidator()
    {
        RuleFor(r => r.TableNumber)
            .InclusiveBetween(1, 20)
            .WithMessage("Il numero del tavolo deve essere tra 1 e 20");

        RuleFor(r => r.ReservationDate)
            .GreaterThan(DateTime.Now)
            .WithMessage("Non puoi prenotare una data nel passato");

        RuleFor(r => r.UserId)
            .NotEmpty()
            .WithMessage("L'utente è obbligatorio");
    }
}
```

📺 APPROFONDIMENTO

Perché `IReservation` e non creare direttamente l'Entity?

In un'architettura *Slice Vertical*, il domain model è il **contratto di business** — quello che rimane stabile mentre il resto cambia.

Senza `IReservation` → Code duplication:

```
// Nel Repository
public async Task<ReservationEntity> GetById(string id) { ... }

// Nel Command Handler
public async Task<ReservationEntity> Create(CreateReservationCommand cmd) { ... }

// Nel Mapper
public ReservationDto ToDto(ReservationEntity entity) { ... }

// Nel frontend Type
export interface Reservation {
  id: string
  userId: string
  // ... ripeti le stesse proprietà ovunque!
}
```

Con **IReservation** → Una sola fonte di verità:

```
// Tutti implementano/usano IReservation
public ReservationEntity : IReservation { } // Implementa l'interfaccia
public ReservationRecord : IReservation { } // Implementa l'interfaccia
public ReservationDto : IReservation { } // Implementa l'interfaccia

// Il frontend usa la stessa struttura
export interface Reservation { ... } // Corrisponde a IReservation
```

Se cambi **IReservation** (aggiungi una proprietà), tutti gli altri oggetti sanno subito che devono cambiare (strong typing).

Perché un **record** e non una **class** ?

```
// ❌ class - mutabile, potrebbe cambiare
public class ReservationRecord
{
  public string Id { get; set; } // Qualcuno potrebbe fare .Id = "altro" - non voluto
}

// ✅ record - immutabile (di default)
public record ReservationRecord(string Id, string UserId, ...)
{
  // Non puoi fare: r.Id = "altro"
  // È garantito che non cambi
}
```

Perché il Validator qui e non nel Command Handler?

Le regole di business (tavolo 1-20, data nel futuro) appartengono al dominio, non alla logica di input. Se decidi di creare una prenotazione da un'API diversa, da un batch job, o da un webhook, le stesse regole devono valere. Il validator nel dominio garantisce coerenza.

Come si collega al resto:

```
IReservation (definito qui)
├── Usato da: Repository, Query/Command Handlers, Mapper
├── Il frontend riceve uno DTO che rispecchia IReservation
└── Frontend crea la sua interfaccia Reservation (TypeScript)
```

Passaggio 2: Creare la Migration e il Seed del Database

Cosa fare

Creare lo schema del database e popolare dati di test:

1. File: **20260329_AddReservationsTable.cs** — Migration EF Core
2. Tabella **Reservations** con colonne secondo **IReservation**
3. Foreign Key: **UserId** → Tabella **Users**
4. Constraint: **UNIQUE(UserId, TableNumber, ReservationDate)** per evitare doppioni
5. Indici su: **UserId**, **ReservationDate**, **Status**
6. File: **SeedReservations.cs** — Seed data
7. Creare 3-4 prenotazioni di test per diversi utenti
8. Mix di stati: **Pending**, **Confirmed**, **Cancelled**

File coinvolti

```
Backend/  
├─ Migrations/  
│   └─ 20260329_AddReservationsTable.cs ← NEW  
├─ Seed/  
│   └─ SeedReservations.cs ← NEW  
└─ Slices/Reservations/  
    └─ Persistence/  
        └─ ReservationConfiguration.cs ← NEW (EF Core mapping)
```

Codice di Esempio

```
// Migrations/20260329_AddReservationsTable.cs
public partial class AddReservationsTable : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.CreateTable(
            name: "Reservations",
            columns: table => new
            {
                Id = table.Column<string>(type: "nvarchar(36)", nullable: false),
                UserId = table.Column<string>(type: "nvarchar(36)", nullable: false),
                TableNumber = table.Column<int>(type: "int", nullable: false),
                ReservationDate = table.Column<DateTime>(type: "datetime2", nullable: false),
                Status = table.Column<int>(type: "int", nullable: false), // enum as int
                CreatedAt = table.Column<DateTime>(type: "datetime2", nullable: false),
                UpdatedAt = table.Column<DateTime>(type: "datetime2", nullable: false),
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_Reservations", x => x.Id);
                table.ForeignKey(
                    name: "FK_Reservations_Users_UserId",
                    column: x => x.UserId,
                    principalTable: "Users",
                    principalColumn: "Id",
                    onDelete: ReferentialAction.Cascade);
                // Evita prenotazioni doppie per lo stesso utente/tavolo/data
                table.UniqueConstraint(
                    name: "UX_Reservations_UserIdTableNumberDate",
                    columns: new[] { "UserId", "TableNumber", "ReservationDate" });
            });

        migrationBuilder.CreateIndex(
            name: "IX_Reservations_UserId",
            table: "Reservations",
            column: "UserId");

        migrationBuilder.CreateIndex(
            name: "IX_Reservations_ReservationDate",
            table: "Reservations",
            column: "ReservationDate");
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropTable(name: "Reservations");
    }
}

// Seed/SeedReservations.cs
public static class ReservationSeeds
{
    public static void SeedReservations(this ModelBuilder modelBuilder)
    {
        var tomorrow = DateTime.UtcNow.AddDays(1).Date.AddHours(19);
        var nextWeek = DateTime.UtcNow.AddDays(7).Date.AddHours(20);

        modelBuilder.Entity<ReservationEntity>().HasData(
            new ReservationEntity
            {
                Id = Guid.NewGuid().ToString(),
                UserId = "user-1", // Assumi che user-1 esista
                TableNumber = 5,
                ReservationDate = tomorrow,
                Status = ReservationStatus.Confirmed,
                CreatedAt = DateTime.UtcNow,
                UpdatedAt = DateTime.UtcNow,
            },
            new ReservationEntity
            {
                Id = Guid.NewGuid().ToString(),
                UserId = "user-2",
                TableNumber = 10,
                ReservationDate = nextWeek,
                Status = ReservationStatus.Pending,
                CreatedAt = DateTime.UtcNow,
                UpdatedAt = DateTime.UtcNow,
            }
        );
    }
}

// Persistence/ReservationConfiguration.cs
public class ReservationConfiguration : IEntityTypeConfiguration<ReservationEntity>
{
    public void Configure(EntityTypeBuilder<ReservationEntity> builder)
    {
        builder.HasKey(r => r.Id);

        builder.Property(r => r.Status)
            .HasConversion<int>() // Salva enum come numero
            .HasDefaultValue(ReservationStatus.Pending);

        builder.HasOne<UserEntity>()
            .WithMany()
            .HasForeignKey(r => r.UserId)
            .OnDelete(DeleteBehavior.Cascade);
    }
}
```

```
builder.HasIndex(r => r.UserId);
builder.HasIndex(r => r.ReservationDate);

builder.HasAlternateKey(r => new { r.UserId, r.TableNumber, r.ReservationDate });
}
```

APPROFONDIMENTO

Perché una Migration separata e non creare direttamente l'Entity?

In un progetto production: - **Storico delle modifiche**: ogni migration è tracciata e reversibile - **Collaborazione**: se due developer modificano il DB, le migration si mergeano - **CI/CD**: le migration vengono applicate automaticamente sui vari ambienti (dev, staging, prod)

```
Migration = "Versione controllata del schema"
           = Git per il database
```

Perché **UNIQUE** (UserId, TableNumber, ReservationDate) ?

Impedisce che lo stesso utente prenoti lo stesso tavolo nella stessa data:

```
✗ Non con questo constraint:
User "Mario" → Table 5 → 2026-04-15 → ✓ (prima prenotazione)
User "Mario" → Table 5 → 2026-04-15 → ✓ (doppione! No, non dovrebbe)

✓ Con constraint:
User "Mario" → Table 5 → 2026-04-15 → ✓ (prima prenotazione)
User "Mario" → Table 5 → 2026-04-15 → ✗ Errore: violato unique constraint
```

Perché gli indici su UserId, ReservationDate, Status?

Le query che farai spesso: - "Tutte le prenotazioni di User X" → indice su **UserId** → veloce ✓ - "Prenotazioni da domani in poi" → indice su **ReservationDate** → veloce ✓ - "Quante prenotazioni sono Pending?" → indice su **Status** → veloce ✓

Senza indici, il DB farebbe una scansione completa (slow).

ReservationEntity vs ReservationRecord?

```
// ReservationRecord (Pass. 1) — Domain Model
// Immutabile, usato nella logica di business
public record ReservationRecord(...) : IReservation

// ReservationEntity (Pass. 2) — EF Core Entity
// Traccia le modifiche per il DB, ha proprietà extra (shadow keys, ecc.)
public class ReservationEntity : IReservation
{
    // EF Core le traccia automaticamente
    public DbSet<ReservationEntity> Reservations { get; set; }
}
```

Sono due cose diverse: - **Record** = ciò che passi nella logica - **Entity** = ciò che EF Core persiste

Passaggio 3: Creare Repository e Query Handler (GET)

Cosa fare

Implementare il livello **data access** e la logica di lettura:

1. File: **IReservationRepository.cs** — Interfaccia del repository
2. Metodi: **GetAllAsync()**, **GetByIdAsync()**, **CreateAsync()**, **UpdateAsync()**, **DeleteAsync()**
3. File: **ReservationRepository.cs** — Implementazione EF Core
4. Usa **DbContext** per interrogare/modificare il DB
5. Nessuna logica di business qui, solo accesso dati
6. File: **GetAllReservationsQuery.cs** — Query MEDIATR
7. Request: eventualmente con filtri (UserId, Status, DateRange)
8. Response: List di **ReservationDto**
9. File: **GetAllReservationsQueryHandler.cs** — Handler MEDIATR
10. Chiama repository, mappa a DTO, ritorna
11. File: **ReservationDto.cs** — Data Transfer Object
12. Stesse proprietà di **IReservation** ma un DTO separato

File coinvolti

```
Backend/Slices/Reservations/  
├─ Repository/  
│   ├── IReservationRepository.cs    ← NEW  
│   └─ ReservationRepository.cs      ← NEW  
├─ Queries/  
│   ├── GetAllReservationsQuery.cs   ← NEW  
│   └─ GetAllReservationsQueryHandler.cs ← NEW  
├─ Dtos/  
│   └─ ReservationDto.cs             ← NEW  
└─ Persistence/  
    └─ Mapper/  
        └─ ReservationMapper.cs      ← NEW
```

Codice di Esempio

```
// 📁 Repository/IReservationRepository.cs
public interface IReservationRepository
{
    Task<IEnumerable<IReservation>> GetAllAsync(string? userId = null);
    Task<IReservation?> GetByIdAsync(string id);
    Task<IReservation> CreateAsync(IReservation reservation);
    Task<IReservation> UpdateAsync(IReservation reservation);
    Task DeleteAsync(string id);
}

// 📁 Repository/ReservationRepository.cs
public class ReservationRepository : IReservationRepository
{
    private readonly AppDbContext _context;

    public ReservationRepository(AppDbContext context) => _context = context;

    public async Task<IEnumerable<IReservation>> GetAllAsync(string? userId = null)
    {
        var query = _context.Reservations.AsQueryable();

        if (!string.IsNullOrEmpty(userId))
            query = query.Where(r => r.UserId == userId);

        var entities = await query.OrderBy(r => r.ReservationDate).ToListAsync();
        return entities.Cast<IReservation>().ToList();
    }

    public async Task<IReservation?> GetByIdAsync(string id)
    {
        var entity = await _context.Reservations.FindAsync(id);
        return entity as IReservation;
    }

    public async Task<IReservation> CreateAsync(IReservation reservation)
    {
        var entity = reservation as ReservationEntity ?? throw new InvalidOperationException();
        _context.Reservations.Add(entity);
        await _context.SaveChangesAsync();
        return entity;
    }

    public async Task<IReservation> UpdateAsync(IReservation reservation)
    {
        var entity = reservation as ReservationEntity ?? throw new InvalidOperationException();
        _context.Reservations.Update(entity);
        await _context.SaveChangesAsync();
        return entity;
    }

    public async Task DeleteAsync(string id)
    {
        var entity = await _context.Reservations.FindAsync(id);
        if (entity == null)
            throw new KeyNotFoundException($"Prenotazione {id} non trovata");

        _context.Reservations.Remove(entity);
        await _context.SaveChangesAsync();
    }
}

// 📁 Queries/GetAllReservationsQuery.cs
public class GetAllReservationsQuery : IRequest<List<ReservationDto>>
{
    public string? UserId { get; set; }
    public ReservationStatus? Status { get; set; }
}

// 📁 Queries/GetAllReservationsQueryHandler.cs
public class GetAllReservationsQueryHandler : IRequestHandler<GetAllReservationsQuery, List<ReservationDto>>
{
    private readonly IReservationRepository _repository;
    private readonly IMapper _mapper;

    public GetAllReservationsQueryHandler(IReservationRepository repository, IMapper mapper)
    {
        _repository = repository;
        _mapper = mapper;
    }

    public async Task<List<ReservationDto>> Handle(GetAllReservationsQuery request, CancellationToken cancellationToken)
    {
        var reservations = await _repository.GetAllAsync(request.UserId);

        var filtered = reservations
            .Where(r => request.Status == null || r.Status == request.Status)
            .ToList();

        return _mapper.Map<List<ReservationDto>>(filtered);
    }
}

// 📁 Dtos/ReservationDto.cs
public class ReservationDto
{
    public string Id { get; set; }
    public string UserId { get; set; }
    public int TableNumber { get; set; }
}
```



```

    public DateTime ReservationDate { get; set; }
    public ReservationStatus Status { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime UpdatedAt { get; set; }
}

// Persistence/Mapper/ReservationMapper.cs
public class ReservationProfile : Profile
{
    public ReservationProfile()
    {
        CreateMap<IReservation, ReservationDto>();
        CreateMap<ReservationEntity, ReservationDto>();
    }
}

```

APPROFONDIMENTO

Fare riferimento al Passaggio 1 per comprendere perché `IReservation` è il contratto.

Perché il Repository pattern?

Repository nasconde i dettagli di come accedi ai dati:

Senza Repository:

```

Query Handler
{
    var data = _context.Reservations -- Conosce EF Core
    .Where(r => r.UserId == userId) -- Conosce la struttura DB
    .ToList();
}

```

! Problema: Se cambi da EF Core a Dapper,
devi cambiare il Query Handler

Con Repository:

```

Query Handler
{
    var data = _repository -- Chiama il repository
    .GetAllAsync(userId); -- Non sa come funziona
}

```

!

```

Repository
{
    // EF Core? Dapper? SQL raw? -- Dettagli qui
    var data = _context.Reservations...
}

```

! Vantaggio: cambi l'implementazione,
il Query Handler non sa nulla

Perché DTO separato da Domain Model?

```

Domain Model (IReservation):
├─ Proprietà di business: UserId, TableNumber, ReservationDate
└─ Regole di validazione: tavolo 1-20, data nel futuro

DTO (ReservationDto):
├─ Stesse proprietà di sopra
└─ + CreatedAt, UpdatedAt (metadati tecnici)

```

Il DTO separa ciò che è dominio (`IReservation`) da ciò che è implementazione tecnica (`createdAt`, `updatedAt`).

Se domani decidi di non esporre `Status` nel frontend, cambi il DTO, non `IReservation`.

Perché GetAllReservationsQuery con MEDIATR?

MEDIATR è un message dispatcher:

```

Senza MEDIATR:
// Nel controller
public async Task<List<ReservationDto>> GetAll(string? userId)
{
    return await _repository.GetAllAsync(userId);
}

Con MEDIATR:
// Nel handler
public async Task<List<ReservationDto>> Handle(GetAllReservationsQuery query, ...)
{
    return await _repository.GetAllAsync(query.UserId);
}

// Nel controller
public async Task<List<ReservationDto>> GetAll(string? userId)
{
    var query = new GetAllReservationsQuery { UserId = userId };
    return await _mediator.Send(query);
}

```

Vantaggio:

```

Logica di filtro/validazione
è nel Handler, non nel Controller
→ facile da testare
→ facile da riutilizzare

```

Passaggio 4: Creare Command Handler per CREATE

Cosa fare

Implementare la logica di **creazione** di una nuova prenotazione:

1. File: **CreateReservationCommand.cs** — Comando MEDIATR
2. Proprietà: **UserId**, **TableNumber**, **ReservationDate**
3. Nessuna validazione qui, solo dati
4. File: **CreateReservationCommandHandler.cs** — Handler MEDIATR
5. Valida il comando (usa **ReservationValidator**)
6. Crea la nuova prenotazione
7. Salva via repository
8. Ritorna il DTO
9. File: **CreateReservationCommandValidator.cs** — FluentValidation
10. Valida gli input (UserId non vuoto, TableNumber range, etc.)

File coinvolti

```

Backend/Slices/Reservations/
├── Commands/
│   ├── CreateReservationCommand.cs      ← NEW
│   ├── CreateReservationCommandValidator.cs ← NEW
│   └── CreateReservationCommandHandler.cs ← NEW

```

Codice di Esempio

```
// Commands/CreateReservationCommand.cs
public class CreateReservationCommand : IRequest<ReservationDto>
{
    public string UserId { get; set; }
    public int TableNumber { get; set; }
    public DateTime ReservationDate { get; set; }
}

// Commands/CreateReservationCommandValidator.cs
public class CreateReservationCommandValidator : AbstractValidator<CreateReservationCommand>
{
    public CreateReservationCommandValidator()
    {
        RuleFor(c => c.UserId)
            .NotEmpty()
            .WithMessage("L'utente è obbligatorio");

        RuleFor(c => c.TableNumber)
            .InclusiveBetween(1, 20)
            .WithMessage("Il numero del tavolo deve essere tra 1 e 20");

        RuleFor(c => c.ReservationDate)
            .GreaterThan(DateTime.Now)
            .WithMessage("Non puoi prenotare una data nel passato");
    }
}

// Commands/CreateReservationCommandHandler.cs
public class CreateReservationCommandHandler : IRequestHandler<CreateReservationCommand, ReservationDto>
{
    private readonly IReservationRepository _repository;
    private readonly IValidator<CreateReservationCommand> _validator;
    private readonly IMapper _mapper;

    public CreateReservationCommandHandler(
        IReservationRepository repository,
        IValidator<CreateReservationCommand> validator,
        IMapper mapper)
    {
        _repository = repository;
        _validator = validator;
        _mapper = mapper;
    }

    public async Task<ReservationDto> Handle(CreateReservationCommand request, CancellationToken cancellationToken)
    {
        // Valida il comando
        var validationResult = await _validator.ValidateAsync(request, cancellationToken);
        if (!validationResult.IsValid)
            throw new ValidationException(validationResult.Errors);

        // Crea il domain model
        var reservation = new ReservationRecord(
            Id: Guid.NewGuid().ToString(),
            UserId: request.UserId,
            TableNumber: request.TableNumber,
            ReservationDate: request.ReservationDate,
            Status: ReservationStatus.Pending,
            CreatedAt: DateTime.UtcNow,
            UpdatedAt: DateTime.UtcNow
        );

        // Salva
        var created = await _repository.CreateAsync(reservation);

        // Ritorna DTO
        return _mapper.Map<ReservationDto>(created);
    }
}
```

 APPROFONDIMENTO

Fare riferimento al [Passaggio 3](#) per comprendere il pattern Repository e MEDIATR.

Perché Command separato da Query?

```
Query (GetAllReservationsQuery):
├─ "Dammi i dati"
├─ Non cambia lo stato
└─ Può essere cacheata, è idempotente

Command (CreateReservationCommand):
├─ "Crea una cosa nuova"
├─ Cambia lo stato del DB
└─ Non è idempotente (ogni volta crea una nuova)

Pattern: CQRS = Command Query Responsibility Segregation
Leggi e scritte sono completamente separate
```

Perché Validator sul Command, se già c'è su IReservation?

```
ReservationValidator (Passaggio 1):
└─ Valida la struttura del dominio
    "Un IReservation DEVE avere tavolo 1-20"

CreateReservationCommandValidator (Passaggio 4):
└─ Valida l'input dall'utente/API
    "Un CreateReservationCommand DEVE avere UserId non vuoto"

Il secondo è più specifico: non tutte le proprietà di una richiesta
sono obbligatorie in un ReservationRecord (ad es., Id viene generato).
```

Flow di un'operazione CREATE:

```
Frontend invia POST /api/reservations
┆
┆ CreateReservationCommand
┆
┆ CreateReservationCommandValidator
┆
┆ Valida? No → ProblemDetails 400
┆
┆ Valida? Si
┆
┆ ReservationValidator (su domain model)
┆
┆ Valida? No → ProblemDetails 400
┆
┆ Valida? Si → _repository.CreateAsync()
┆
┆ INSERT in database
┆
┆ Mappa a ReservationDto
┆
┆ Ritorna 201 Created + DTO
┆
┆ Frontend riceve il nuovo oggetto
```

Passaggio 5: Creare Command Handler per UPDATE

Cosa fare

Implementare la logica di **modifica** di una prenotazione esistente:

1. File: **UpdateReservationCommand.cs** — Comando MEDIATR
2. Proprietà: **Id**, **TableNumber**, **ReservationDate**, **Status**
3. Nota: **UserId** non è modificabile (stabilito alla creazione)
4. File: **UpdateReservationCommandValidator.cs** — FluentValidation
5. Valida gli input come CREATE ma con **Id** obbligatorio
6. File: **UpdateReservationCommandHandler.cs** — Handler MEDIATR
7. Recupera la prenotazione esistente
8. Verifica che l'utente loggato sia il proprietario (TODO: verifica permessi)
9. Aggiorna le proprietà
10. Salva

File coinvolti

```
Backend/Slices/Reservations/
├─ Commands/
│   └─ UpdateReservationCommand.cs      ← NEW
│   └─ UpdateReservationCommandValidator.cs ← NEW
│   └─ UpdateReservationCommandHandler.cs ← NEW
```

Codice di Esempio

```
// Commands/UpdateReservationCommand.cs
public class UpdateReservationCommand : IRequest<ReservationDto>
{
    public string Id { get; set; }
    public int TableNumber { get; set; }
    public DateTime ReservationDate { get; set; }
    public ReservationStatus Status { get; set; }
}

// Commands/UpdateReservationCommandValidator.cs
public class UpdateReservationCommandValidator : AbstractValidator<UpdateReservationCommand>
{
    public UpdateReservationCommandValidator()
    {
        RuleFor(c => c.Id)
            .NotEmpty()
            .WithMessage("L'ID è obbligatorio");

        RuleFor(c => c.TableNumber)
            .InclusiveBetween(1, 20)
            .WithMessage("Il numero del tavolo deve essere tra 1 e 20");

        RuleFor(c => c.ReservationDate)
            .GreaterThan(DateTime.Now)
            .WithMessage("Non puoi prenotare una data nel passato");

        RuleFor(c => c.Status)
            .IsInEnum()
            .WithMessage("Lo stato della prenotazione non è valido");
    }
}

// Commands/UpdateReservationCommandHandler.cs
public class UpdateReservationCommandHandler : IRequestHandler<UpdateReservationCommand, ReservationDto>
{
    private readonly IReservationRepository _repository;
    private readonly IValidator<UpdateReservationCommand> _validator;
    private readonly IMapper _mapper;

    public UpdateReservationCommandHandler(
        IReservationRepository repository,
        IValidator<UpdateReservationCommand> validator,
        IMapper mapper)
    {
        _repository = repository;
        _validator = validator;
        _mapper = mapper;
    }

    public async Task<ReservationDto> Handle(UpdateReservationCommand request, CancellationToken cancellationToken)
    {
        // Valida il comando
        var validationResult = await _validator.ValidateAsync(request, cancellationToken);
        if (!validationResult.IsValid)
            throw new ValidationException(validationResult.Errors);

        // Recupera la prenotazione esistente
        var existing = await _repository.GetByIdAsync(request.Id);
        if (existing == null)
            throw new KeyNotFoundException($"Prenotazione {request.Id} non trovata");

        // TODO: Verifica che l'utente loggato sia il proprietario
        // var currentUser = _httpContextAccessor.HttpContext?.User.FindFirst("sub")?.Value;
        // if (existing.UserId != currentUser)
        //     throw new UnauthorizedAccessException("Non puoi modificare questa prenotazione");

        // Crea il domain model aggiornato
        var updated = new ReservationRecord(
            Id: existing.Id,
            UserId: existing.UserId, // Non modificabile
            TableNumber: request.TableNumber,
            ReservationDate: request.ReservationDate,
            Status: request.Status,
            CreatedAt: existing.CreatedAt,
            UpdatedAt: DateTime.UtcNow
        );

        // Salva
        var result = await _repository.UpdateAsync(updated);

        // Ritorna DTO
        return _mapper.Map<ReservationDto>(result);
    }
}
```

 APPROFONDIMENTO

Fare riferimento ai Passaggi 3 e 4 per comprendere Repository, MEDIATR, e il pattern Command.

Aggiunta rispetto a CREATE:

1. **Recupero della risorsa esistente** → `GetByIdAsync()`
2. **Verifica di ownership** → solo il proprietario può modificare
3. **Id immutabile** → non puoi cambiare a chi appartiene

Passaggio 6: Creare Command Handler per DELETE

Cosa fare

Implementare la logica di **eliminazione** di una prenotazione:

1. File: **DeleteReservationCommand.cs** — Comando MEDIATR
2. Proprietà: **Id** della prenotazione da eliminare
3. File: **DeleteReservationCommandValidator.cs** — FluentValidation
4. Valida che **Id** non sia vuoto
5. File: **DeleteReservationCommandHandler.cs** — Handler MEDIATR
6. Recupera la prenotazione
7. Verifica ownership
8. Elimina dal repository

File coinvolti

```
Backend/Slices/Reservations/  
├── Commands/  
│   ├── DeleteReservationCommand.cs      ← NEW  
│   ├── DeleteReservationCommandValidator.cs ← NEW  
│   └── DeleteReservationCommandHandler.cs ← NEW
```

Codice di Esempio

```
// Commands/DeleteReservationCommand.cs  
public class DeleteReservationCommand : IRequest<Unit>  
{  
    public string Id { get; set; }  
}  
  
// Commands/DeleteReservationCommandValidator.cs  
public class DeleteReservationCommandValidator : AbstractValidator<DeleteReservationCommand>  
{  
    public DeleteReservationCommandValidator()  
    {  
        RuleFor(c => c.Id)  
            .NotEmpty()  
            .WithMessage("L'ID è obbligatorio");  
    }  
}  
  
// Commands/DeleteReservationCommandHandler.cs  
public class DeleteReservationCommandHandler : IRequestHandler<DeleteReservationCommand, Unit>  
{  
    private readonly IReservationRepository _repository;  
    private readonly IValidator<DeleteReservationCommand> _validator;  
  
    public DeleteReservationCommandHandler(  
        IReservationRepository repository,  
        IValidator<DeleteReservationCommand> validator)  
    {  
        _repository = repository;  
        _validator = validator;  
    }  
  
    public async Task<Unit> Handle(DeleteReservationCommand request, CancellationToken cancellatic  
    {  
        // Valida il comando  
        var validationResult = await _validator.ValidateAsync(request, cancellationTok  
        if (!validationResult.IsValid)  
            throw new ValidationException(validationResult.Errors);  
  
        // Recupera la prenotazione  
        var existing = await _repository.GetByIdAsync(request.Id);  
        if (existing == null)  
            throw new KeyNotFoundException($"Prenotazione {request.Id} non trovata");  
  
        // TODO: Verifica ownership (come in UPDATE)  
  
        // Elimina  
        await _repository.DeleteAsync(request.Id);  
  
        return Unit.Value;  
    }  
}
```

APPROFONDIMENTO

Fare riferimento al **Passaggio 5** per comprendere il pattern UPDATE. DELETE è analogo ma senza aggiornamenti.

Differenza tra DELETE e UPDATE:

```
UPDATE: Leggi → Modifica → Salva
        Necessita di nuovi valori

DELETE: Leggi → Elimina
        Non ha nuovi valori, solo conferma
        Ritorna Unit (nessun dato), non ReservationDto
```

Passaggio 7: Creare Endpoints API

Cosa fare

Esporre gli **endpoint HTTP** che il frontend chiamerà:

1. File: **ReservationsController.cs** — Controller ASP.NET Core
2. Route: **/api/reservations**
3. Endpoint:
 - **GET /api/reservations** → GetAllReservationsQuery
 - **GET /api/reservations/{id}** → GetReservationByIdQuery (NEW)
 - **POST /api/reservations** → CreateReservationCommand
 - **PUT /api/reservations/{id}** → UpdateReservationCommand
 - **DELETE /api/reservations/{id}** → DeleteReservationCommand
4. File: **GetReservationByIdQuery.cs** e handler — NEW
5. Query per ottenere una singola prenotazione

File coinvolti

```
Backend/Slices/Reservations/
├── Queries/
│   ├── GetReservationByIdQuery.cs      ← NEW
│   └── GetReservationByIdQueryHandler.cs ← NEW
├── Endpoints/
│   └── ReservationsController.cs      ← NEW
```

Codice di Esempio

```
// 📄 Queries/GetReservationByIdQuery.cs
public class GetReservationByIdQuery : IRequest<ReservationDto>
{
    public string Id { get; set; }
}

// 📄 Queries/GetReservationByIdQueryHandler.cs
public class GetReservationByIdQueryHandler : IRequestHandler<GetReservationByIdQuery, ReservationDto>
{
    private readonly IReservationRepository _repository;
    private readonly IMapper _mapper;

    public GetReservationByIdQueryHandler(IReservationRepository repository, IMapper mapper)
    {
        _repository = repository;
        _mapper = mapper;
    }

    public async Task<ReservationDto> Handle(GetReservationByIdQuery request, CancellationToken cancellationToken)
    {
        var reservation = await _repository.GetByIdAsync(request.Id);
        if (reservation == null)
            throw new KeyNotFoundException($"Prenotazione {request.Id} non trovata");

        return _mapper.Map<ReservationDto>(reservation);
    }
}

// 📄 Endpoints/ReservationsController.cs
[ApiController]
[Route("api/[controller]")]
[Authorize] // Tutte le route richiedono autenticazione
public class ReservationsController : ControllerBase
{
    private readonly IMediator _mediator;

    public ReservationsController(IMediator mediator)
    {
        _mediator = mediator;
    }

    /// <summary>
    /// GET /api/reservations
    /// Ottiene tutte le prenotazioni, opzionalmente filtrate per utente
    /// </summary>
    [HttpGet]
    [ProducesResponseType(typeof(List<ReservationDto>), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status400BadRequest)]
    public async Task<ActionResult<List<ReservationDto>>> GetAll(
        [FromQuery] string? userId,
        [FromQuery] ReservationStatus? status)
    {
        var query = new GetAllReservationsQuery { UserId = userId, Status = status };
        var result = await _mediator.Send(query);
        return Ok(result);
    }

    /// <summary>
    /// GET /api/reservations/{id}
    /// Ottiene una prenotazione per ID
    /// </summary>
    [HttpGet("{id}")]
    [ProducesResponseType(typeof(ReservationDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status404NotFound)]
    public async Task<ActionResult<ReservationDto>> GetById(string id)
    {
        var query = new GetReservationByIdQuery { Id = id };
        var result = await _mediator.Send(query);
        return Ok(result);
    }

    /// <summary>
    /// POST /api/reservations
    /// Crea una nuova prenotazione
    /// </summary>
    [HttpPost]
    [ProducesResponseType(typeof(ReservationDto), StatusCodes.Status201Created)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status400BadRequest)]
    public async Task<ActionResult<ReservationDto>> Create([FromBody] CreateReservationCommand command)
    {
        var result = await _mediator.Send(command);
        return CreatedAtAction(nameof(GetById), new { id = result.Id }, result);
    }

    /// <summary>
    /// PUT /api/reservations/{id}
    /// Aggiorna una prenotazione
    /// </summary>
    [HttpPut("{id}")]
    [ProducesResponseType(typeof(ReservationDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status400BadRequest)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status404NotFound)]
    public async Task<ActionResult<ReservationDto>> Update(string id, [FromBody] UpdateReservationCommand command)
    {
        if (command.Id != id)
            return BadRequest("L'ID nel body non corrisponde all'ID nella URL");

        var result = await _mediator.Send(command);
    }
}
```



```

        return Ok(result);
    }

    /// <summary>
    /// DELETE /api/reservations/{id}
    /// Elimina una prenotazione
    /// </summary>
    [HttpDelete("{id}")]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status404NotFound)]
    public async Task<IActionResult> Delete(string id)
    {
        var command = new DeleteReservationCommand { Id = id };
        await _mediator.Send(command);
        return NoContent();
    }
}

```

APPROFONDIMENTO

Struttura HTTP vs MEDIATR:

```

HTTP Layer (ReservationsController):
└─ Riceve la richiesta HTTP
└─ Parsing della URL e body
└─ Response HTTP (200, 201, 404, etc.)

MEDIATR Layer (Handlers):
└─ Logica di business
└─ Validazione
└─ Accesso ai dati

Separazione netta: Controller non fa logica, Handler non sa dell'HTTP

```

Codici HTTP:

```

GET /api/reservations          → 200 OK (con lista)
GET /api/reservations/{id}     → 200 OK (con oggetto) o 404 Not Found

POST /api/reservations         → 201 Created (con Location header)
PUT /api/reservations/{id}     → 200 OK (con oggetto aggiornato)
DELETE /api/reservations/{id}  → 204 No Content

```

[Authorize] vs logica di ownership:

```

[Authorize] nel Controller:
└─ "Devi essere loggato per accedere"

Verifica ownership nel Handler (TODO nei passaggi precedenti):
└─ "Devi essere il proprietario di questa risorsa per modificarla"

Entrambi necessari:
1. [Authorize] protegge l'accesso alle API
2. Ownership protegge da accessi incrociati (Mario non vede le prenotazioni di Luigi)

```



FRONTEND — 7 Passaggi

Passaggio 8: Creare Types API (Reservation.ts)

Cosa fare

Aggiungere le interfacce **TypeScript** che corrispondono ai DTO backend:

1. File: **api.types.ts** — Aggiungere al file esistente
2. Interfaccia **Reservation** (corrisponde a **ReservationDto** backend)
3. Enum **ReservationStatus**
4. Interfacce per le request: **CreateReservationRequest**, **UpdateReservationRequest**

File coinvolti

```

apps/frontend/src/types/
└─ api.types.ts ← MODIFICA (aggiungi al file esistente)

```

Codice di Esempio

```
// 📄 types/api.types.ts - Aggiungi queste interfacce

// === RESERVATIONS ===

export enum ReservationStatus {
  Pending = 0,
  Confirmed = 1,
  Cancelled = 2,
}

export interface Reservation {
  id: string
  userId: string
  tableNumber: number
  reservationDate: string // ISO 8601 datetime
  status: ReservationStatus
  createdAt: string
  updatedAt: string
}

export interface CreateReservationRequest {
  userId: string
  tableNumber: number
  reservationDate: string
}

export interface UpdateReservationRequest {
  tableNumber: number
  reservationDate: string
  status: ReservationStatus
}
```

APPROFONDIMENTO

Types TypeScript vs Backend DTO:

Il backend invia `ReservationDto`, il frontend lo riceve come `Reservation`:

Backend		Frontend
ReservationDto	====>	Reservation
├ id		├ id
├ userId		├ userId
├ tableNumber		├ tableNumber
├ reservationDate		├ reservationDate
├ status		├ status
├ createdAt		├ createdAt
└ updatedAt		└ updatedAt

Sono identici strutturalmente, ma in due linguaggi diversi.
TypeScript non ha accesso al backend, quindi li definisce qui.

Enum ReservationStatus:

```
// Nel backend è un enum C#:
public enum ReservationStatus { Pending = 0, Confirmed = 1, ... }

// Viene serializzato come numero (0, 1, 2) quando percorre HTTP
// Nel frontend lo ricevi come numero, quindi definiamo lo stesso enum

// Quando mandi al backend:
await api.post('/api/reservations', {
  userId: 'user-1',
  tableNumber: 5,
  reservationDate: '2026-04-15T19:00:00Z',
  status: ReservationStatus.Pending // ← numero 0
})

// Il backend sa che 0 = Pending perché ha lo stesso enum
```

Passaggio 9: Creare Service (reservations.service.ts)

Cosa fare

Creare il **service** che fa le chiamate HTTP:

1. File: `services/reservations.service.ts` — NEW
2. Metodi: `getAll()`, `getById()`, `create()`, `update()`, `delete()`
3. Usa l'istanza `api` (axios configurato)

File coinvolti

```
apps/frontend/src/services/  
└─ reservations.service.ts ← NEW
```

Codice di Esempio

```
// 📄 services/reservations.service.ts  
import { api } from '@plugins/axios'  
import type {  
  Reservation,  
  CreateReservationRequest,  
  UpdateReservationRequest,  
  PagedResponse,  
} from '@types/api.types'  
  
export const reservationsService = {  
  getAll: (params?: { userId?: string }) =>  
    api.get<PagedResponse<Reservation>>('/api/reservations', { params }).then(r => r.data),  
  
  getById: (id: string) =>  
    api.get<Reservation>(`/api/reservations/${id}`).then(r => r.data),  
  
  create: (data: CreateReservationRequest) =>  
    api.post<Reservation>('/api/reservations', data).then(r => r.data),  
  
  update: (id: string, data: UpdateReservationRequest) =>  
    api.put<Reservation>(`/api/reservations/${id}`, data).then(r => r.data),  
  
  delete: (id: string) =>  
    api.delete(`/api/reservations/${id}`),  
}
```

APPROFONDIMENTO

Fare riferimento al Passaggio 8 — il Service usa i tipi definiti lì.

Perché un Service separato?

Senza Service (❌):

```
Store  
{  
  async fetchAll() {  
    const res =  
      await axios  
        .get(...) ← Conosce l'URL  
  }  
}
```

Problema: il Store sa come fare HTTP

Con Service (✅):

```
Store  
{  
  async fetchAll() {  
    const res =  
      await service  
        .getAll() ← Usa il service  
  } ← Service sa come fare HTTP  
}
```

|
↓

```
Service  
{  
  getAll: () =>  
    api.get(...) ← Dettagli HTTP qui  
}
```

Vantaggio: cambi l'API base URL, cambi solo il Service.
Lo store non sa nulla dell'HTTP.

Parametri opzionali:

```
// Nel service:  
getAll: (params?: { userId?: string }) =>  
  api.get<PagedResponse<Reservation>>('/api/reservations', { params })  
  
// Nel store:  
reservations.value = await reservationsService.getAll({ userId: 'user-1' })  
  
// Si trasforma in:  
GET /api/reservations?userId=user-1
```

Passaggio 10: Creare Store Pinia (reservations.store.ts)

Cosa fare

Creare lo **state management** con Pinia:

1. File: `stores/reservations.store.ts` — NEW
2. Stato: `reservations[]` , `selectedReservation` , `loading` , `error` , `totalCount`
3. Azioni: `fetchAll()` , `fetchById()` , `create()` , `update()` , `delete()`
4. Computati: eventualmente `myReservations` (solo dell'utente loggato)

File coinvolti

```
apps/frontend/src/stores/  
└─ reservations.store.ts  ← NEW
```

Codice di Esempio

```
// 📁 stores/reservations.store.ts
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import { reservationsService } from '@/services/reservations.service'
import type { Reservation, CreateReservationRequest, UpdateReservationRequest } from '@/types/api'
import { useAuthStore } from './auth.store'
import { useI18n } from './plugins/i18n'

export const useReservationsStore = defineStore('reservations', () => {
  // --- Stato ---
  const reservations = ref<Reservation[]>([])
  const selectedReservation = ref<Reservation | null>(null)
  const loading = ref(false)
  const error = ref<string | null>(null)
  const totalCount = ref(0)

  // --- Computati ---
  const myReservations = computed(() => {
    const authStore = useAuthStore()
    if (!authStore.user) return []
    return reservations.value.filter(r => r.userId === authStore.user?.id)
  })

  // --- Azioni ---
  async function fetchAll(userId?: string) {
    loading.value = true
    error.value = null
    try {
      const res = await reservationsService.getAll({ userId })
      reservations.value = res.items
      totalCount.value = res.totalCount
    } catch {
      error.value = useI18n.global.t('errors.loadReservations')
    } finally {
      loading.value = false
    }
  }

  async function fetchById(id: string) {
    loading.value = true
    error.value = null
    try {
      selectedReservation.value = await reservationsService.getById(id)
    } catch {
      error.value = useI18n.global.t('errors.loadReservation')
    } finally {
      loading.value = false
    }
  }

  async function create(data: CreateReservationRequest) {
    loading.value = true
    error.value = null
    try {
      const newReservation = await reservationsService.create(data)
      reservations.value.push(newReservation)
      return newReservation
    } catch (e) {
      error.value = useI18n.global.t('errors.createReservation')
      throw e
    } finally {
      loading.value = false
    }
  }

  async function update(id: string, data: UpdateReservationRequest) {
    loading.value = true
    error.value = null
    try {
      const updated = await reservationsService.update(id, data)
      const index = reservations.value.findIndex(r => r.id === id)
      if (index >= 0) {
        reservations.value[index] = updated
      }
      if (selectedReservation.value?.id === id) {
        selectedReservation.value = updated
      }
      return updated
    } catch (e) {
      error.value = useI18n.global.t('errors.updateReservation')
      throw e
    } finally {
      loading.value = false
    }
  }

  async function deleteReservation(id: string) {
    loading.value = true
    error.value = null
    try {
      await reservationsService.delete(id)
      reservations.value = reservations.value.filter(r => r.id !== id)
      if (selectedReservation.value?.id === id) {
        selectedReservation.value = null
      }
    } catch (e) {
      error.value = useI18n.global.t('errors.deleteReservation')
      throw e
    }
  }
})
```

```

    } finally {
      loading.value = false
    }
  }

  return {
    // Stato
    reservations,
    selectedReservation,
    loading,
    error,
    totalCount,
    // Computati
    myReservations,
    // Azioni
    fetchAll,
    fetchById,
    create,
    update,
    deleteReservation,
  }
})

```

APPROFONDIMENTO

Fare riferimento al Passaggio 9 — lo Store usa il Service.

Pinia Store = State Management:

Componente Vue:

- └ "Ho bisogno dei dati"
- └ Non chiama axios direttamente
- └ Chiama lo Store

Store (Pinia):

- └ Tiene i dati in memoria
- └ Quando chiedi dati, li ritorna
- └ Quando chiedi di creare/modificare, chiama il Service
- └ Aggiorna lo stato locale in base alla risposta

Schema:

```

graph TD
    Component[Component] -- "└ chiede dati" --> Store[Store]
    Store -- "└ chiama" --> Service[Service]
    Service -- "└ fa HTTP API" --> API[API]
    
```

└ Una sola istanza per tutta l'app

Azioni (actions) vs Computed:

```

// Computed: derivato dallo stato, NON fa operazioni async
const myReservations = computed(() => {
  // Filtra lo stato locale
  return reservations.value.filter(r => r.userId === authStore.user?.id)
})

// Action: operazione, può essere async
async function fetchAll(userId?: string) {
  // Chiama il service, che fa HTTP
  // Aggiorna lo stato
}

```

Gestione errori:

Ogni azione ha try/catch:

```

try {
  // Fai l'operazione
} catch (e) {
  error.value = i18n.global.t('errors.xxx') // Messaggio tradotto
  throw e // Rilancia per il componente che la chiama
} finally {
  loading.value = false // Sempre, anche se errore
}

```

Passaggio 11: Creare Page Lista (ReservationsPage.vue)

Cosa fare

Creare la **pagina principale** che mostra la lista delle prenotazioni:

1. File: `pages/reservations/ReservationsPage.vue` — NEW
2. Usa Vuetify `v-data-table` per la lista
3. Colonne: `id` , `tableNumber` , `reservationDate` , `status` , `azioni`
4. Azioni: click sulla riga → apri detail, bottone "Nuovo" → apri create dialog
5. Filtri: filtro per UserId (opzionale), search

File coinvolti

```
apps/frontend/src/pages/reservations/  
└─ ReservationsPage.vue ← NEW
```

Codice di Esempio

```

<!-- 📄 pages/reservations/ReservationsPage.vue -->
<script setup lang="ts">
import { ref, computed, onMounted } from 'vue'
import { useRouter } from 'vue-router'
import { useI18n } from 'vue-i18n'
import { useDisplay } from 'vuetify'
import { useReservationsStore } from '@stores/reservations.store'
import { usePermission } from '@composables/usePermission'
import { useToastStore } from '@stores/toast.store'
import { useBreadcrumb } from '@composables/useBreadcrumb'
import ConfirmDialog from '@components/shared/ConfirmDialog.vue'
import CreateReservationDialog from '@components/reservations/CreateReservationDialog.vue'

const { t } = useI18n()
const { mobile } = useDisplay()
const router = useRouter()
const store = useReservationsStore()
const { can } = usePermission()
const toast = useToastStore()
const { setBreadcrumb } = useBreadcrumb()

// --- Stato ---
const search = ref('')
const page = ref(1)
const itemsPerPage = ref(25)
const deleteDialog = ref(false)
const deleteTarget = ref<string | null>(null)
const deleteLoading = ref(false)
const createDialog = ref(false)

// --- Computati ---
const filtered = computed(() => {
  if (!search.value) return store.reservations
  const s = search.value.toLowerCase()
  return store.reservations.filter(r =>
    r.id.toLowerCase().includes(s) ||
    r.tableNumber.toString().includes(s) ||
    r.userId.toLowerCase().includes(s)
  )
})

const columns = computed(() => [
  { title: t('reservations.columnId'), key: 'id', sortable: true },
  { title: t('reservations.columnUser'), key: 'userId', sortable: true },
  { title: t('reservations.columnTable'), key: 'tableNumber', sortable: true },
  { title: t('reservations.columnDate'), key: 'reservationDate', sortable: true },
  { title: t('reservations.columnStatus'), key: 'status', sortable: true },
  { title: t('common.actions'), key: 'actions', sortable: false },
])

// --- Funzioni ---
function openCreateDialog() {
  createDialog.value = true
}

function openDetail(id: string) {
  router.push({ name: 'reservation-detail', params: { id } })
}

function openDeleteDialog(id: string) {
  deleteTarget.value = id
  deleteDialog.value = true
}

async function confirmDelete() {
  if (!deleteTarget.value) return

  deleteLoading.value = true
  try {
    await store.deleteReservation(deleteTarget.value)
    toast.success(t('reservations.deleted'))
    deleteDialog.value = false
  } catch {
    toast.error(store.error || t('errors.deleteReservation'))
  } finally {
    deleteLoading.value = false
  }
}

async function onCreateSuccess() {
  createDialog.value = false
  toast.success(t('reservations.created'))
  await store.fetchAll()
}

// --- Lifecycle ---
onMounted(() => {
  setBreadcrumb([
    { text: t('common.home'), href: '/' },
    { text: t('reservations.title'), disabled: true },
  ])
  store.fetchAll()
})
</script>

<template>
<v-container>
  <v-row>

```



```

<v-col>
  <div class="d-flex align-center justify-space-between mb-6">
    <h1 class="text-h5">{{ t('reservations.title') }}</h1>

    <v-btn
      v-if="can('reservations.create')"
      color="primary"
      prepend-icon="mdi-plus"
      @click="openCreateDialog"
    >
      {{ t('reservations.createButton') }}
    </v-btn>
  </div>

  <!-- Search -->
  <v-text-field
    v-model="search"
    :label="t('common.search')"
    variant="outlined"
    density="compact"
    prepend-inner-icon="mdi-magnify"
    clearable
    class="mb-4"
    :style="{ maxWidth: mobile ? '100%' : '400px' }"
  />

  <!-- Data Table -->
  <v-data-table
    :items="filtered"
    :columns="columns"
    :loading="store.loading"
    :page.sync="page"
    :items-per-page="itemsPerPage"
    hover
  >
    <template #item.reservationDate="{ item }">
      {{ new Date(item.reservationDate).toLocaleString() }}
    </template>

    <template #item.status="{ item }">
      <v-chip
        :color="
          item.status === 0 ? 'warning' : item.status === 1 ? 'success' : 'error'
        "
        text-color="white"
        size="small"
      >
        {{ t('reservations.status.${item.status}') }}
      </v-chip>
    </template>

    <template #item.actions="{ item }">
      <v-btn
        variant="text"
        size="x-small"
        icon="mdi-eye"
        @click="openDetail(item.id)"
      />
      <v-btn
        v-if="can('reservations.delete')"
        variant="text"
        size="x-small"
        icon="mdi-delete"
        color="error"
        @click="openDeleteDialog(item.id)"
      />
    </template>

    <template #bottom>
      <v-divider />
      <div class="text-center py-2">
        {{ t('common.total') }}: {{ store.totalCount }}
      </div>
    </template>
  </v-data-table>
</v-col>
</v-row>
</v-container>

<!-- Delete Dialog -->
<ConfirmDialog
  v-model="deleteDialog"
  :title="t('reservations.deleteTitle')"
  :message="t('reservations.deleteConfirm')"
  :loading="deleteLoading"
  @confirm="confirmDelete"
/>

<!-- Create Dialog -->
<CreateReservationDialog
  v-model="createDialog"
  @success="onCreateSuccess"
/>
</template>

```

APPROFONDIMENTO

v-data-table vs v-table:

```
v-table:
└─ Tabella semplice, senza sort/paginate
└─ Usa se hai pochi dati

v-data-table:
└─ Tabella con sort, paginate, filtri
└─ Usa se hai molti dati o devi ordinare

Nel nostro caso usiamo v-data-table perché le prenotazioni possono essere tante.
```

Colonne compute:

```
const columns = computed(() => [
  { title: t('reservations.columnId'), key: 'id', sortable: true },
  // ...
])

// Perché computed? Perché se cambia la lingua t()),
// le colonne si aggiornano automaticamente
```

Template scoped slots:

```
<template #item.status="{ item }">
  <v-chip :color="...">
    {{ t(`reservations.status.${item.status}`) }}
  </v-chip>
</template>

<!-- Personalizza il rendering della colonna "status"
      Mostra un chip colorato al posto di un testo semplice -->
```

Permessi (v-if can()):

```
<v-btn v-if="can('reservations.create')" ...>
  Nuovo
</v-btn>

<!-- Solo se l'utente loggato ha il permesso 'reservations.create'
      il bottone è visibile -->
```

Passaggio 12: Creare Page Dettaglio (ReservationDetailPage.vue)

Cosa fare

Creare la pagina di modifica di una prenotazione:

1. File: `pages/reservations/ReservationDetailPage.vue` — NEW
2. Recupera la prenotazione dal store per ID (dalla URL)
3. Mostra i dati in forma di sola lettura o editabile (dipende da te)
4. Bottone "Modifica" → aprì dialog di update
5. Breadcrumb: Home > Prenotazioni > ID

File coinvolti

```
apps/frontend/src/pages/reservations/
└─ ReservationDetailPage.vue — NEW
```

Codice di Esempio

```
<!-- 📄 pages/reservations/ReservationDetailPage.vue -->
<script setup lang="ts">
import { onMounted, computed } from 'vue'
import { useRouter, useRoute } from 'vue-router'
import { useI18n } from 'vue-i18n'
import { useReservationsStore } from '@/stores/reservations.store'
import { usePermission } from '@/composables/usePermission'
import { useBreadcrumb } from '@/composables/useBreadcrumb'
import EditReservationDialog from '@/components/reservations/EditReservationDialog.vue'
import { ref } from 'vue'

const { t } = useI18n()
const router = useRouter()
const route = useRoute()
const store = useReservationsStore()
const { can } = usePermission()
const { setBreadcrumb } = useBreadcrumb()

const editDialog = ref(false)
const reservation = computed(() => store.selectedReservation)

onMounted(async () => {
  const id = route.params.id as string
  await store.fetchById(id)

  setBreadcrumb([
    { text: t('common.home'), href: '/' },
    { text: t('reservations.title'), href: '/reservations' },
    { text: id, disabled: true },
  ])
})
</script>

<template>
  <v-container>
    <v-row v-if="store.loading">
      <v-col>
        <v-skeleton-loader type="article" />
      </v-col>
    </v-row>

    <v-row v-else-if="!reservation">
      <v-col>
        <v-alert type="error" :text="t('errors.notFound')" />
      </v-col>
    </v-row>

    <v-row v-else>
      <v-col>
        <v-card>
          <v-card-title>
            {{ t('reservations.detail') }} - {{ reservation.id }}
          </v-card-title>

          <v-card-text>
            <v-list lines="two">
              <v-list-item :title="t('reservations.columnId')" :subtitle="reservation.id" />
              <v-list-item :title="t('reservations.columnUser')" :subtitle="reservation.userId" />
              <v-list-item :title="t('reservations.columnTable')" :subtitle="String(reservation.tableId)" />
              <v-list-item
                :title="t('reservations.columnDate')"
                :subtitle="new Date(reservation.reservationDate).toLocaleString()"
              />
              <v-list-item
                :title="t('reservations.columnStatus')"
                >
                <template #subtitle>
                  <v-chip
                    :color="reservation.status === 0 ? 'warning' : reservation.status === 1 ? 'success' : 'error'
                    text-color="white"
                    size="small"
                  >
                    {{ t('reservations.status.' + reservation.status) }}
                  </v-chip>
                </template>
              </v-list-item>
              <v-list-item
                :title="t('common.createdAt')"
                :subtitle="new Date(reservation.createdAt).toLocaleString()"
              />
              <v-list-item
                :title="t('common.updatedAt')"
                :subtitle="new Date(reservation.updatedAt).toLocaleString()"
              />
            </v-list>
          </v-card-text>

          <v-card-actions>
            <v-btn variant="tonal" @click="$router.back()">
              {{ t('common.back') }}
            </v-btn>

            <v-spacer />

            <v-btn
              v-if="can('reservations.update')"
              color="primary"
              variant="elevated"
            >
              {{ t('common.update') }}
            </v-btn>
          </v-card-actions>
        </v-card>
      </v-col>
    </v-row>
  </v-container>
</template>
```

```
        @click="editDialog = true"
      >
        {{ t('common.edit') }}
      </v-btn>
    </v-card-actions>
  </v-card>
</v-col>
</v-row>
</v-container>

<EditReservationDialog
  v-model="editDialog"
  :reservation="reservation!"
  @success="() => editDialog = false"
/>
</template>
```

APPROFONDIMENTO

Fare riferimento al **Passaggio 11** — dettaglio è simile a lista ma mostra un singolo elemento.

Differenze tra List e Detail:

```
ReservationsPage (Lista):
└─ v-data-table con N righe
└─ Click → vai a detail

ReservationDetailPage (Dettaglio):
└─ Un solo elemento, proprietà per proprietà
└─ Leggi da store (store.selectedReservation)
└─ Bottone modifica → apri dialog
```

Breadcrumb:

```
setBreadcrumb([
  { text: 'Home', href: '/' },
  { text: 'Prenotazioni', href: '/reservations' },
  { text: id, disabled: true }, // Disable l'ultimo, non è un link
])

// Mostra: Home > Prenotazioni > ID
```

Passaggio 13: Creare Dialog Componenti (Create e Edit)

Cosa fare

Creare i **componenti dialog** per creazione e modifica:

1. File: **components/reservations/CreateReservationDialog.vue** — NEW
2. Form con campi: UserId, TableNumber, ReservationDate
3. Validazione client con v-form
4. Bottone Salva → chiama store.create()
5. Emette evento **@success** al completamento
6. File: **components/reservations/EditReservationDialog.vue** — NEW
7. Come CreateReservationDialog ma per update
8. Riempie i campi con i dati della prenotazione
9. Chiama store.update()

File coinvolti

```
apps/frontend/src/components/reservations/
├─ CreateReservationDialog.vue  ← NEW
└─ EditReservationDialog.vue    ← NEW
```

Codice di Esempio

```

<!-- components/reservations/CreateReservationDialog.vue -->
<script setup lang="ts">
import { ref, computed } from 'vue'
import { useI18n } from 'vue-i18n'
import { useReservationsStore } from '@stores/reservations.store'
import { useAuthStore } from '@stores/auth.store'
import { useToastStore } from '@stores/toast.store'
import type { CreateReservationRequest } from '@types/api.types'

const { t } = useI18n()
const store = useReservationsStore()
const authStore = useAuthStore()
const toast = useToastStore()

const props = defineProps({
  modelValue: {
    type: Boolean,
    required: true,
  },
})

const emit = defineEmits<{
  'update:modelValue': [boolean]
  'success': []
}>()

// --- Form ---
const form = ref<CreateReservationRequest>({
  userId: authStore.user?.id || '',
  tableNumber: 1,
  reservationDate: new Date().toISOString().split('T')[0],
})

const formRef = ref()
const loading = ref(false)

const isOpen = computed({
  get: () => props.modelValue,
  set: (value) => emit('update:modelValue', value),
})

const tableNumberRules = [
  (v: number) => v >= 1 && v <= 20 || t('reservations.tableNumberError'),
]

const dateRules = [
  (v: string) => new Date(v) > new Date() || t('reservations.dateError'),
]

// --- Funzioni ---
async function handleSubmit() {
  if (!formRef.value) return

  const isValid = await formRef.value.validate()
  if (!isValid.valid) return

  loading.value = true
  try {
    await store.create({
      ...form.value,
      reservationDate: new Date(form.value.reservationDate).toISOString(),
    })
    toast.success(t('reservations.created'))
    isOpen.value = false
    emit('success')
  } catch {
    toast.error(store.error || t('errors.createReservation'))
  } finally {
    loading.value = false
  }
}

function handleClose() {
  isOpen.value = false
}
</script>

<template>
<v-dialog v-model="isOpen" width="500">
  <v-card>
    <v-card-title>{{ t('reservations.createTitle') }}</v-card-title>

    <v-card-text>
      <v-form ref="formRef" class="pt-4">
        <!-- User (read-only, already set) -->
        <v-text-field
          :model-value="authStore.user?.email"
          :label="t('reservations.columnUser')"
          disabled
          class="mb-4"
        />

        <!-- Table Number -->
        <v-text-field
          v-model.number="form.tableNumber"
          type="number"
          :label="t('reservations.columnTable')"
          :rules="tableNumberRules"

```

```
      min="1"
      max="20"
      required
      class="mb-4"
    />

    <!-- Reservation Date -->
    <v-text-field
      v-model="form.reservationDate"
      type="date"
      :label="t('reservations.columnDate')"
      :rules="dateRules"
      required
      class="mb-4"
    />
  </v-form>
</v-card-text>

<v-card-actions>
  <v-spacer />
  <v-btn variant="tonal" @click="handleClose">
    {{ t('common.cancel') }}
  </v-btn>
  <v-btn
    color="primary"
    variant="elevated"
    :loading="loading"
    @click="handleSubmit"
  >
    {{ t('common.save') }}
  </v-btn>
</v-card-actions>
</v-card>
</v-dialog>
</template>
```

```

<!-- 📁 components/reservations/EditReservationDialog.vue -->
<script setup lang="ts">
import { ref, computed, watch } from 'vue'
import { useI18n } from 'vue-i18n'
import { useReservationsStore } from '@stores/reservations.store'
import { useToastStore } from '@stores/toast.store'
import type { Reservation, UpdateReservationRequest, ReservationStatus } from '@types/api.types'

const { t } = useI18n()
const store = useReservationsStore()
const toast = useToastStore()

const props = defineProps({
  modelValue: {
    type: Boolean,
    required: true,
  },
  reservation: {
    type: Object as () => Reservation,
    required: true,
  },
})

const emit = defineEmits<{
  'update:modelValue': [boolean]
  'success': []
}>()

// --- Form ---
const form = ref<UpdateReservationRequest>({
  tableNumber: 1,
  reservationDate: '',
  status: 0,
})

const formRef = ref()
const loading = ref(false)

const isOpen = computed({
  get: () => props.modelValue,
  set: (value) => emit('update:modelValue', value),
})

const tableNumberRules = [
  (v: number) => v >= 1 && v <= 20 || t('reservations.tableNumberError'),
]

const dateRules = [
  (v: string) => new Date(v) > new Date() || t('reservations.dateError'),
]

// --- Watchers ---
watch(() => props.reservation, () => {
  form.value = {
    tableNumber: props.reservation.tableNumber,
    reservationDate: new Date(props.reservation.reservationDate).toISOString().split('T')[0],
    status: props.reservation.status as ReservationStatus,
  }
}, { immediate: true })

// --- Funzioni ---
async function handleSubmit() {
  if (!formRef.value) return

  const isValid = await formRef.value.validate()
  if (!isValid.valid) return

  loading.value = true
  try {
    await store.update(props.reservation.id, {
      ...form.value,
      reservationDate: new Date(form.value.reservationDate).toISOString(),
    })
    toast.success(t('reservations.updated'))
    isOpen.value = false
    emit('success')
  } catch {
    toast.error(store.error || t('errors.updateReservation'))
  } finally {
    loading.value = false
  }
}

function handleClose() {
  isOpen.value = false
}
</script>

<template>
<v-dialog v-model="isOpen" width="500">
  <v-card>
    <v-card-title>{{ t('reservations.editTitle') }}</v-card-title>

    <v-card-text>
      <v-form ref="formRef" class="pt-4">
        <!-- ID (read-only) -->
        <v-text-field
          :model-value="reservation.id"
          :label="t('reservations.columnId')"
          disabled
          class="mb-4"

```

```

/>

<!-- Table Number -->
<v-text-field
  v-model.number="form.tableNumber"
  type="number"
  :label="t('reservations.columnTable')"
  :rules="tableNumberRules"
  min="1"
  max="20"
  required
  class="mb-4"
/>

<!-- Reservation Date -->
<v-text-field
  v-model="form.reservationDate"
  type="date"
  :label="t('reservations.columnDate')"
  :rules="dateRules"
  required
  class="mb-4"
/>

<!-- Status -->
<v-select
  v-model="form.status"
  :label="t('reservations.columnStatus')"
  :items="[
    { title: t('reservations.status.0'), value: 0 },
    { title: t('reservations.status.1'), value: 1 },
    { title: t('reservations.status.2'), value: 2 },
  ]"
  required
  class="mb-4"
/>
</v-form>
</v-card-text>

<v-card-actions>
  <v-spacer />
  <v-btn variant="tonal" @click="handleClose">
    {{ t('common.cancel') }}
  </v-btn>
  <v-btn
    color="primary"
    variant="elevated"
    :loading="loading"
    @click="handleSubmit"
  >
    {{ t('common.save') }}
  </v-btn>
</v-card-actions>
</v-card>
</v-dialog>
</template>

```

APPROFONDIMENTO

Props e Emit per il dialogo:

```

// Props: Cosa il genitore passa al dialog
props = {
  modelValue: boolean, // Controlla se il dialog è aperto
  reservation: Reservation, // (solo in Edit) la prenotazione da modificare
}

// Emit: Cosa il dialog comunica al genitore
emit = {
  'update:modelValue': [boolean], // v-model two-way binding
  'success': [], // Avvisa che l'operazione è riuscita
}

// Nel genitore (ReservationsPage.vue):
<CreateReservationDialog
  v-model="createDialog" // - Aggiorna createDialog quando dialog si chiude
  @success="onCreateSuccess" // - Ascolta l'evento di successo
/>

```

Validazione client vs server:

```

Client (nel dialog):
└─ Validazione veloce (regex, range)
└─ UX immediato: l'utente vede l'errore subito

Server (backend):
└─ Validazione di business (UNIQUE, autorizzazioni, logica)
└─ Sicurezza: non fidarti del client

Flow:
1. User compila form
2. Dialog valida (client) → se errore, mostra messaggio
3. User clicca Salva → Dialog invia POST
4. Server valida (server) → se errore, ritorna 400
5. Dialog cattura l'errore → mostra messaggio toast

```


Watch per aggiornare il form:

```
watch(() => props.reservation, () => {
  form.value = {
    tableNumber: props.reservation.tableNumber,
    // ... popola il form con i valori della prenotazione
  }
}, { immediate: true })

// Quando il componente riceve una nuova prenotazione,
// il form si aggiorna automaticamente
```

Passaggio 14: Creare i18n e Router

Cosa fare

Integrare la nuova sezione nel sistema di traduzione e routing:

1. File: **locales/it.ts** — MODIFICA
2. Aggiungere chiavi: `reservations.title`, `reservations.createButton`, `reservations.created`, `reservations.status.*`, etc.
3. File: **locales/en.ts** — MODIFICA
4. Stesse chiavi ma in inglese
5. File: **router/index.ts** — MODIFICA
6. Aggiungere 2 route:
 - `/reservations` → `ReservationsPage`
 - `/reservations/:id` → `ReservationDetailPage`
7. Aggiungere meta: `permission: 'reservations.read'`

File coinvolti

```
apps/frontend/src/
├── locales/
│   ├── it.ts      ← MODIFICA
│   └── en.ts      ← MODIFICA
└── router/
    └── index.ts    ← MODIFICA
```

Codice di Esempio

```
// 📄 locales/it.ts - Aggiungi a questo file

export default {
  // ... resto del file ...

  reservations: {
    title: 'Prenotazioni Tavolo',
    createButton: 'Nuova Prenotazione',
    createTitle: 'Crea Prenotazione',
    editTitle: 'Modifica Prenotazione',
    detail: 'Dettaglio Prenotazione',
    created: 'Prenotazione creata con successo',
    updated: 'Prenotazione aggiornata con successo',
    deleted: 'Prenotazione eliminata con successo',
    columnId: 'ID',
    columnUser: 'Utente',
    columnTable: 'Numero Tavolo',
    columnDate: 'Data Prenotazione',
    columnStatus: 'Stato',
    deleteTitle: 'Elimina Prenotazione',
    deleteConfirm: 'Sei sicuro di voler eliminare questa prenotazione?',
    tableNumberError: 'Il numero del tavolo deve essere tra 1 e 20',
    dateError: 'La data deve essere nel futuro',
    status: {
      0: 'In Sospeso',
      1: 'Confermata',
      2: 'Annullata',
    },
  },
},

errors: {
  loadReservations: 'Errore nel caricamento delle prenotazioni',
  loadReservation: 'Errore nel caricamento della prenotazione',
  createReservation: 'Errore nella creazione della prenotazione',
  updateReservation: 'Errore nell\'aggiornamento della prenotazione',
  deleteReservation: 'Errore nell\'eliminazione della prenotazione',
},
}
```

```
// 📄 locales/en.ts - Stesse chiavi in inglese

export default {
  // ... rest of file ...

  reservations: {
    title: 'Table Reservations',
    createButton: 'New Reservation',
    createTitle: 'Create Reservation',
    editTitle: 'Edit Reservation',
    detail: 'Reservation Details',
    created: 'Reservation created successfully',
    updated: 'Reservation updated successfully',
    deleted: 'Reservation deleted successfully',
    columnId: 'ID',
    columnUser: 'User',
    columnTable: 'Table Number',
    columnDate: 'Reservation Date',
    columnStatus: 'Status',
    deleteTitle: 'Delete Reservation',
    deleteConfirm: 'Are you sure you want to delete this reservation?',
    tableNumberError: 'Table number must be between 1 and 20',
    dateError: 'Date must be in the future',
    status: {
      0: 'Pending',
      1: 'Confirmed',
      2: 'Cancelled',
    },
  },
},

  errors: {
    loadReservations: 'Error loading reservations',
    loadReservation: 'Error loading reservation',
    createReservation: 'Error creating reservation',
    updateReservation: 'Error updating reservation',
    deleteReservation: 'Error deleting reservation',
  },
}
```

```
// 📄 router/index.ts - Aggiungi le route

const routes = [
  // ... route esistenti ...

  {
    path: '/reservations',
    name: 'reservations',
    component: () => import('@/pages/reservations/ReservationsPage.vue'),
    meta: {
      requiresAuth: true,
      permission: 'reservations.read',
      title: 'Prenotazioni',
    },
  },
  {
    path: '/reservations/:id',
    name: 'reservation-detail',
    component: () => import('@/pages/reservations/ReservationDetailPage.vue'),
    meta: {
      requiresAuth: true,
      permission: 'reservations.read',
      title: 'Dettaglio Prenotazione',
    },
  },
],

// ... resto delle route ...
]
```

APPROFONDIMENTO

i18n: Perché separare le stringhe?

❌ Senza i18n (hardcoded):
`<h1>Prenotazioni Tavolo</h1>` // Se l'utente cambia lingua, rimane "Prenotazioni"

✅ Con i18n:
`<h1>{{ t('reservations.title') }}</h1>` // Se l'utente cambia lingua, si aggiorna

Vue-i18n gestisce automaticamente la lingua in base alla preferenza dell'utente:

```
// Nel main.ts
app.use(i18n) // Inietta i18n in tutta l'app

// Nel componente
const { t } = useI18n()
{{ t('reservations.title') }} // Legge da locales/it.ts o en.ts
```

Meta della route:

```
meta: {
  requiresAuth: true,      // Richiede autenticazione
  permission: 'reservations.read', // Richiede questo permesso
  title: 'Prenotazioni',   // Usato per il breadcrumb/tab
}

// Nel navigation guard globale (nel router/index.ts):
router.beforeEach((to, from, next) => {
  // Verifica che to.meta.requiresAuth sia true
  // Verifica che l'utente loggato abbia to.meta.permission
  // Se no, redirect a /login
  next()
})
```

Lazy loading delle pagine:

```
// ✅ Preferito:
component: () => import('@/pages/reservations/ReservationsPage.vue')

// Le pagine non vengono caricate finché non le visiti
// Il bundle iniziale è più piccolo
// Solo quando vai a /reservations la pagina viene caricata
```



Checklist Finale

Quando avrai completato tutti i 14 passaggi:

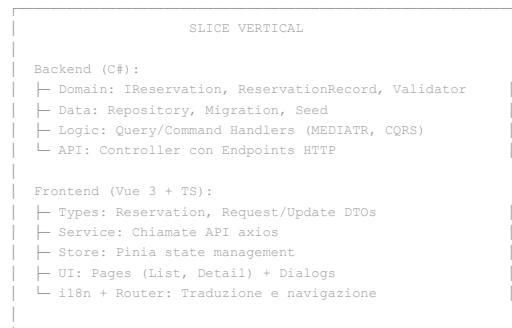
Backend: - [] `IReservation`, `ReservationRecord`, `ReservationStatus`, `ReservationValidator` creati - [] Migration database creata e applicata - [] Seed data inserito - [] `IReservationRepository` e `ReservationRepository` implementati - [] `GetAllReservationsQuery` e handler creati - [] `GetReservationByIdQuery` e handler creati - [] `CreateReservationCommand` e handler creati - [] `UpdateReservationCommand` e handler creati - [] `DeleteReservationCommand` e handler creati - [] `ReservationsController` con tutti gli endpoint creato - [] Validatori `FluentValidation` aggiunti - [] Test unitari per Repository, Handlers, Validators (opzionale ma consigliato)

Frontend: - [] Interfacce `Reservation`, `CreateReservationRequest`, `UpdateReservationRequest` aggiunte a `api.types.ts` - [] `ReservationStatus` enum creato - [] `reservationsService.ts` creato - [] `reservationsStore` (Pinia) creato - [] `ReservationsPage.vue` creato - [] `ReservationDetailPage.vue` creato - [] `CreateReservationDialog.vue` creato - [] `EditReservationDialog.vue` creato - [] i18n: stringhe aggiunte a `it.ts` e `en.ts` - [] Router: route aggiunte

Testing: - [] Test E2E: creare → modificare → eliminare prenotazione - [] Test validazione client (form dialog) - [] Test validazione server (API)



Riassunto Architettura



La **Slice Vertical** garantisce che: - ✓ Tutto ciò che serve per "Prenotazioni" è in una slice - ✓ Se aggiungi un'altra slice (es. "Tavoli"), non si mischiano - ✓ Il dominio è il cuore (`IReservation`), tutto il resto dipende da lui - ✓ Backend e Frontend sono simmetrici (stesso concetto, linguaggi diversi)